

- END-TO-END GUIDE

QE Automation Framework

A TypeScript Playwright framework built on Page Object Model + Component Object Model, dependency-injected fixtures, environment-driven config, and Allure/HTML reporting. This is the complete walkthrough — every feature, the steps to use it, and what it actually looks like on screen.

01

What you're working with

The shape of the framework before you touch a single command.

This repo automates a browser application end to end: UI flows via Page Objects, API calls via a typed client, and everything wired together through Playwright fixtures so a test never constructs its own dependencies. Every run produces two independent reports — Playwright's own HTML report, and a much richer Allure report with named steps, screenshots, and video evidence attached automatically.

Page Object Model

UI locators and actions live in `src/pages/`, never inline in a test.

Fixtures

Every page, service, and helper is injected — tests destructure what they need instead of calling `new`.

Multi-environment

Four config tiers (`qa` / `stage` / `uat` / `prod`) switched with one env var.

13 test disciplines

From unit tests to chaos tests, each with a working reference script.

AI-assisted

Five Copilot chatmodes, two auto-loaded skills, and an MCP server for programmatic control.

Quick start

Five commands from a clean clone to a passing suite.

1 Install dependencies

Also wires up the Husky pre-commit hook (lint-staged runs ESLint + Prettier on every commit).

```
npm install
```

2 Install browsers

Downloads Chromium, Firefox, and WebKit with their OS dependencies.

```
npm run install:browsers
```

3 Configure an environment

Copy the template and fill in real values. `.env.qa` already ships with safe local defaults pointed at the bundled demo app.

```
cp .env.example .env.qa
```

4 Run the suite

`npm test` starts the bundled demo server automatically via Playwright's `webServer` option — nothing to run in a second terminal.

```
npm test
```

5 Open the report

```
npx playwright show-report reports/html
```

No real target app required to start. A minimal demo app ships in `demo/` — a login form, a dashboard, and an animated component — so the smoke/sanity/e2e suites and every sample script run immediately, with zero external dependencies.

Architecture

Where things live, and why.

`src/pages/` sits directly under `src/` because it's what tests touch most. Everything else — the framework "engine" — lives under `src/core/`, grouped by role:

`src/core/ui`

BasePage, reusable Components, Locators — everything that touches the raw Playwright Page API.

`src/core/api`

ApiClient, business Services, optional multi-database clients.

`src/core/runtime`

Fixtures, GlobalSetup/Teardown, console + failure capture hooks.

`src/core/data`

Test data, Fluent builders, domain models, enums.

`src/core/config`

The `Config` singleton, constants, Winston logger.

`src/core/shared`

Interfaces, exceptions, `*Utils` static helpers, validations.

The rule of thumb

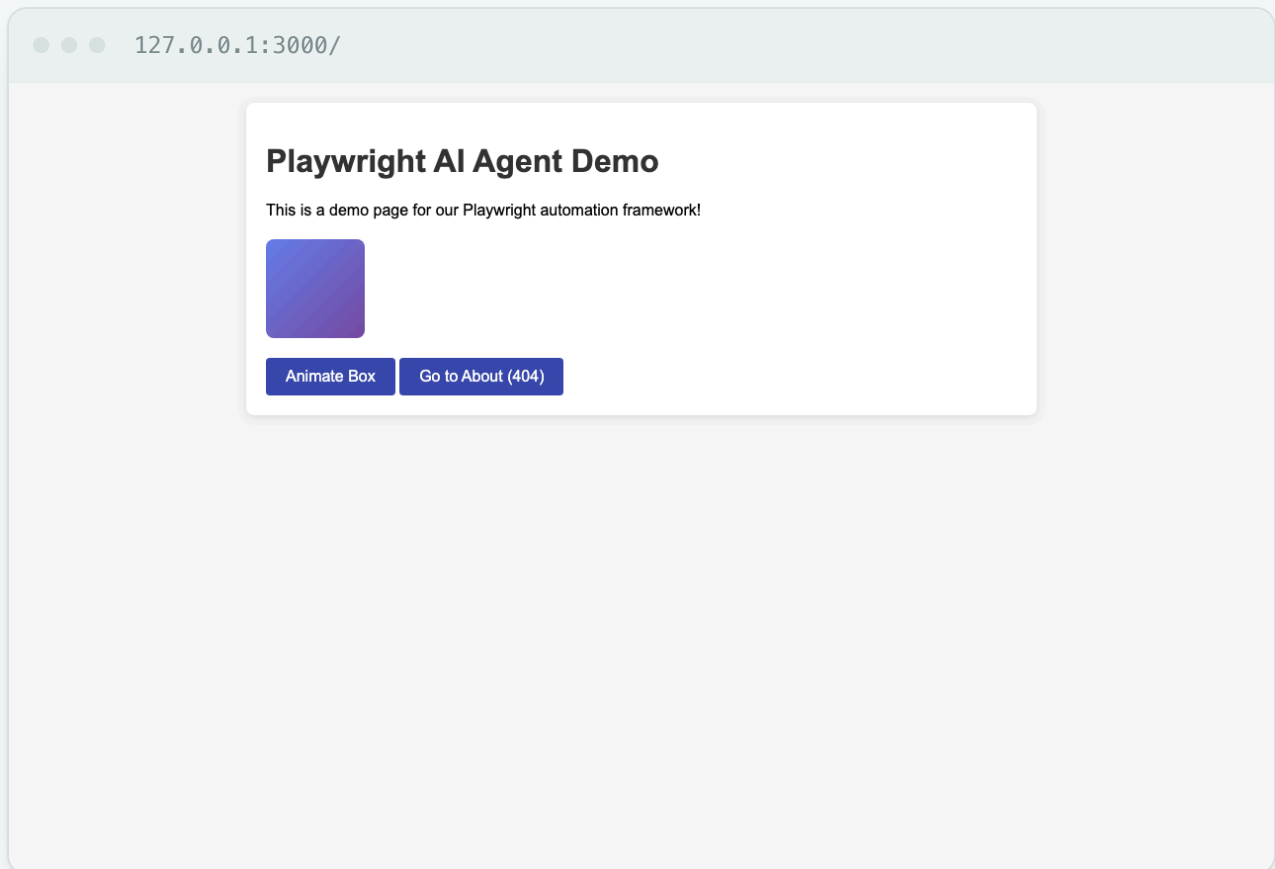
- Tests only do **Arrange / Act / Assert**, using fixtures — never `new PageObject(page)`.
- Page objects only do locators, actions, and navigation — **zero assertions**.
- Assertions live in `ValidationHelpers` or a plain `expect()` in the test.

- Nothing is hardcoded — URLs, credentials, and timeouts always route through `Config` or `src/core/config/constants/` .

The bundled demo app

A real target to point every example in this guide at.

Served from `demo/` by a tiny static server (`src/core/tools/dev-server.js`) on **`http://127.0.0.1:3000`**. Playwright starts and stops it automatically around every test run — you never launch it by hand unless you're exploring manually with `npm run dev:server`.



Home — the target for `DemoPage` and the smoke suite.

127.0.0.1:3000/login

Playwright AI Agent Demo Home

Sign in

Username

not-a-real-user@example.com

Password

Login

Invalid username or password

Login, mid-validation — the target for `LoginPage` .

Running tests

Every command you'll actually type.

By category

COMMAND	RUNS
<code>npm test</code>	Everything, across all five browser projects
<code>npm run test:smoke</code>	Critical-path smoke tests
<code>npm run test:sanity</code>	Quick PR sanity checks
<code>npm run test:regression</code>	Full regression suite
<code>npm run test:e2e</code>	End-to-end user flows
<code>npm run test:api</code>	API-only tests

Browser & environment

COMMAND	EFFECT
<code>npx playwright test --project=chromium</code>	One browser only
<code>BROWSER=firefox npm test</code>	Pin the whole run to one engine
<code>ENVIRONMENT=stage npm test</code>	Load <code>.env.stage</code> instead of <code>.env.qa</code>

Debugging

MODE	COMMAND
Watch it run headed	<code>npm run test:headed</code>
Step through with the Inspector	<code>npm run test:debug</code>
Interactive UI mode	<code>npm run test:ui</code>

Five browser projects run by default: **chromium**, **firefox**, **webkit**, plus **Mobile Chrome** and **Mobile Safari** emulation — configured once in `BrowserUtils.buildProjects()`, not repeated per test.

Writing a new test

The full loop: page object → fixture → spec.

1 Extend BasePage

Locators + actions + navigation only — no assertions, ever.

```
export class DemoPage extends BasePage {
  private readonly animateButton: Locator;

  constructor(page: Page) {
    super(page);
    this.animateButton = this.page.getByRole('button', { name: /animate
  }

  async clickAnimateButton(): Promise<void> {
    await this.click(this.animateButton);
  }
}
```

2 Register it as a fixture

In `src/core/runtime/fixtures/fixtures.ts` — this is what makes it injectable instead of hand-instantiated.

```
demoPage: async ({ page }, use) => {
  await use(new DemoPage(page));
},
```

3 Write the spec

Import from the fixtures file — never from `@playwright/test` directly. That single import is what wires in console/network logging and automatic failure capture.

```
import { test, expect } from '../src/core/runtime/fixtures/fixtures';

test('should click animate button', async ({ demoPage }) => {
  await demoPage.goToDemo();
  await demoPage.clickAnimateButton();
  await expect(demoPage.getAnimatedBox()).toBeVisible();
});
```

4

Run it

```
npx playwright test tests/smoke/demo-smoke.spec.ts --headed
```

13 sample scripts, one per discipline

`tests/zsampleScript/` — a working, passing reference for every kind of test this framework supports.

Each file is a real spec, not pseudo-code — it runs against the demo app as part of a plain `npm test`, and demonstrates the pattern in miniature.

```
npx playwright test tests/zsampleScript # all 13
npx playwright test tests/zsampleScript/security-tests.sample.spec.ts # one ca
```

Unit

01

Pure functions, no browser — parsing, validation, timeout math.

[unit-tests.sample.spec.ts](#)

Integration

02

Multi-step workflows across components.

[integration-tests.sample.spec.ts](#)

Performance

03

Load time, First Contentful Paint, resource count.

[performance-tests.sample.spec.ts](#)

Security

04

Blocked auth, XSS injection, header validation.

[security-tests.sample.spec.ts](#)

Validation

05

Format checks, length limits, malicious patterns.

[validation-tests.sample.spec.ts](#)

Mock

06

Stubbed failures, slow responses, dead services.

[mock-tests.sample.spec.ts](#)

Interop

07

CSS/ES6 feature support, viewport, touch.

[interop-tests.sample.spec.ts](#)

Accessibility

08

Axe scan, keyboard-only nav, focus order.

[accessibility-tests.sample.spec.ts](#)

Resilience

09

Asset failures, partial outages, error states.

[resilience-tests.sample.spec.ts](#)

Network-resilience

10

Offline, throttled, dropped connections.

[network-resilience-tests.sample.spec.ts](#)

i18n

11

Language attributes, direction, pluralization.

[i18n-tests.sample.spec.ts](#)

E2E

12

Full critical-path journey, entirely via POM.

[e2e-tests.sample.spec.ts](#)

Chaos

13

Concurrent users, partial failures, random delays.

[chaos-tests.sample.spec.ts](#)

Reporting

Two independent reports come out of every run — a fast native one, and a detailed one.

Playwright's own HTML report

Zero setup — every run writes it. Best for a quick pass/fail scan right after a local run.

● ● ● reports/html — `npx playwright show-report`

Project: chromium 7/24/2026, 6:07:16 AM Total time: 5.0s

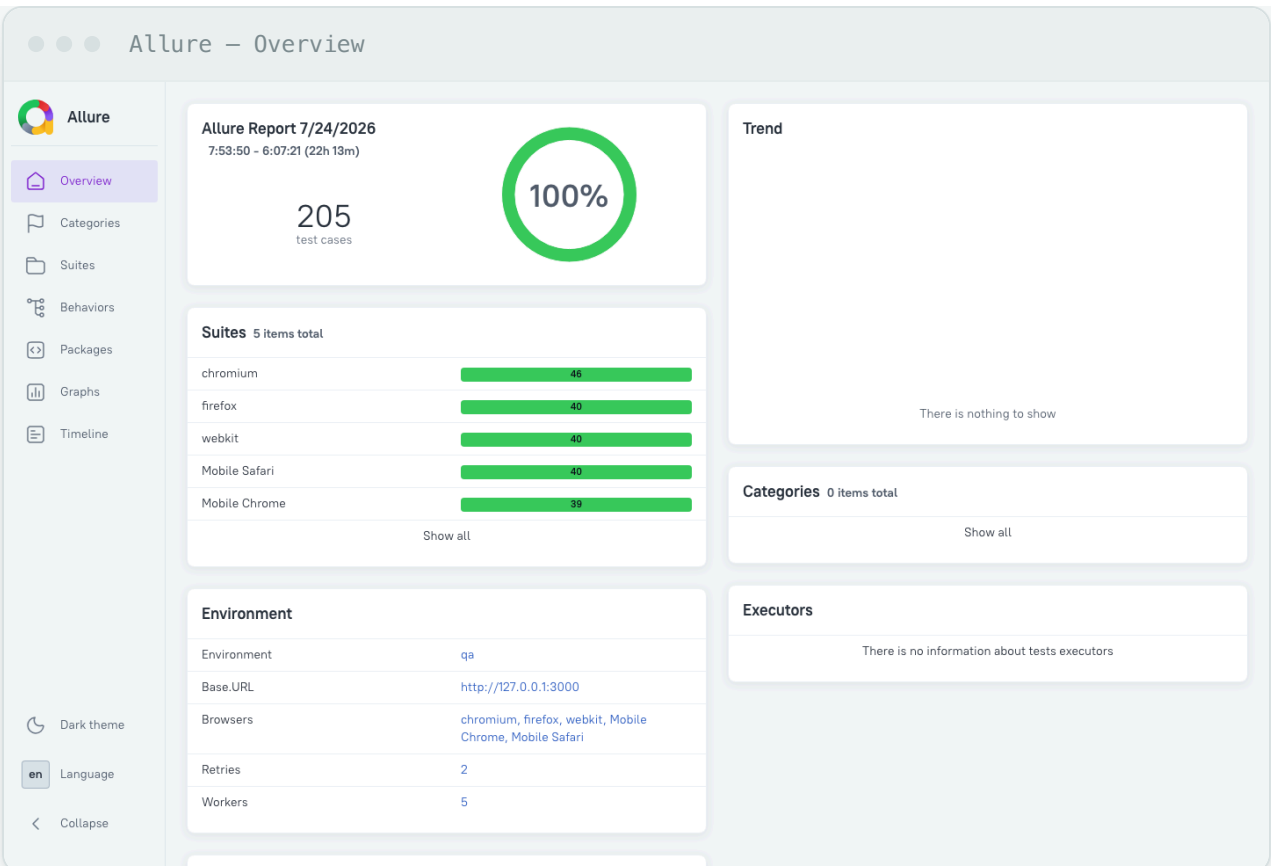
Spec File	Test Name	Browser	Duration
smoke/demo-smoke.spec.ts	✓ Demo Page - Smoke Tests › should load demo page successfully	chromium	295ms
	smoke/demo-smoke.spec.ts:4		
smoke/demo-smoke.spec.ts	✓ Demo Page - Smoke Tests › should click animate button	chromium	356ms
	smoke/demo-smoke.spec.ts:13		
zsampleScript/accessibility-tests.sample.spec.ts	✓ Sample - Accessibility Tests › page has no automatically detectable a11y violations (Axe accessibility checks)	chromium	349ms
	zsampleScript/accessibility-tests.sample.spec.ts:25		
	✓ Sample - Accessibility Tests › interactive elements are reachable via keyboard alone (keyboard-only navigation)	chromium	306ms
zsampleScript/accessibility-tests.sample.spec.ts	zsampleScript/accessibility-tests.sample.spec.ts:35		
	✓ Sample - Accessibility Tests › focus order is sequential and does not get trapped (focus order)	chromium	553ms
zsampleScript/accessibility-tests.sample.spec.ts	zsampleScript/accessibility-tests.sample.spec.ts:48		
	zsampleScript/chaos-tests.sample.spec.ts		
zsampleScript/chaos-tests.sample.spec.ts	✓ Sample - Chaos Tests › multiple concurrent users can load the app at the same time (multiple users)	chromium	196ms
	zsampleScript/chaos-tests.sample.spec.ts:14		
zsampleScript/chaos-tests.sample.spec.ts	✓ Sample - Chaos Tests › one user surviving a backend outage while another succeeds (DB/backend failures)	chromium	171ms
	zsampleScript/chaos-tests.sample.spec.ts:41		
zsampleScript/chaos-tests.sample.spec.ts	✓ Sample - Chaos Tests › requests completing in a random order do not corrupt page state (random delays)	chromium	94ms
	zsampleScript/chaos-tests.sample.spec.ts:62		
zsampleScript/e2e-tests.sample.spec.ts	✓ Sample - E2E Tests › critical path: a user attempts to sign in end-to-end via the POM	chromium	351ms

Grouped by spec file, with duration and browser project per test.

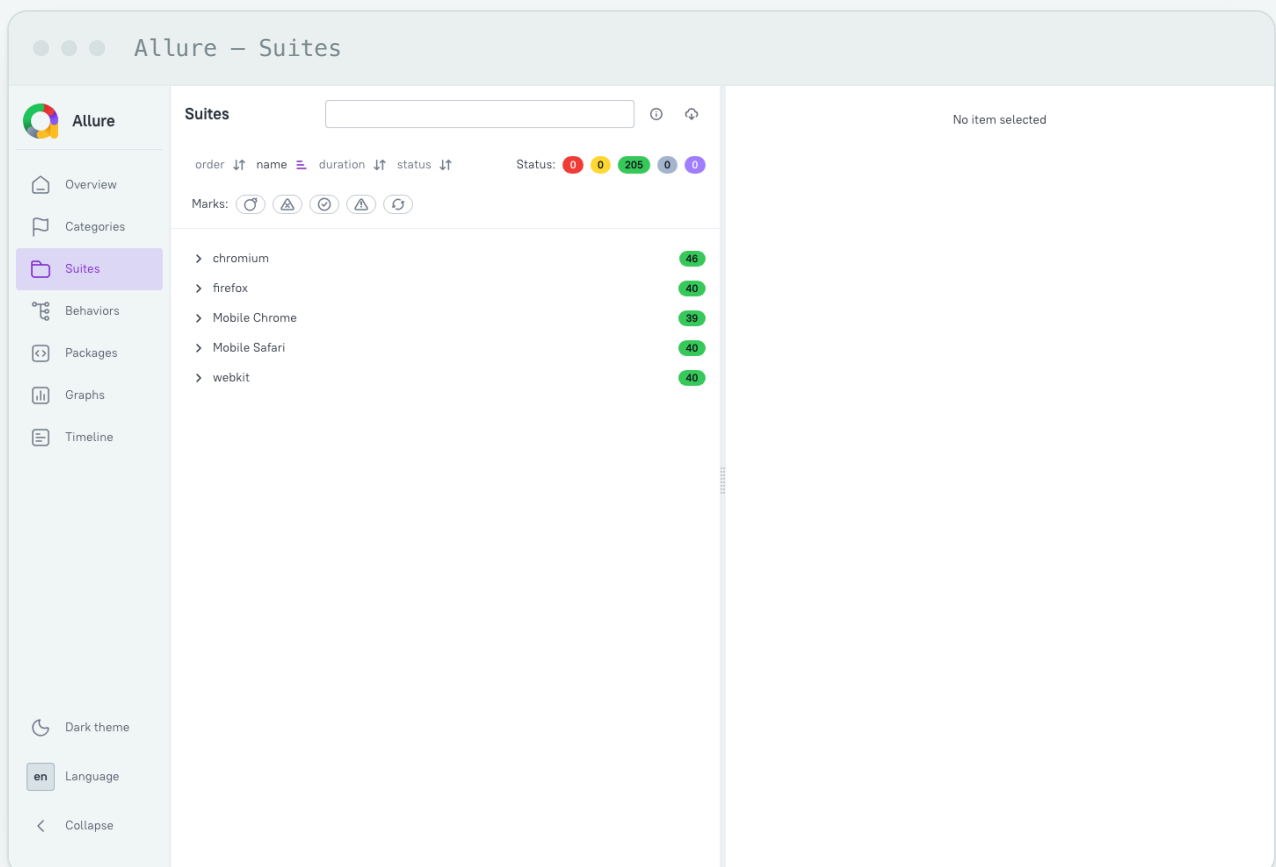
Allure — the detailed report

Raw results are written automatically on every run (the `allure-playwright` reporter is already registered). Generating and opening the static site is a separate step:

```
npm run allurereport # generate + open in one command
# - or, separately -
npm run allure:generate
npm run allure:open
```



The Environment panel is populated automatically by `GlobalSetup.ts` — every report says exactly which `.env` tier, browsers, and retry count produced it.



Every browser project gets its own suite tree — nothing to configure.

Named steps and evidence, on every run

Not just on failure — `AllureUtils` attaches proof for every test, pass or fail.

Playwright's own screenshot/video capture is **failure-only** by config. `AllureUtils` (`src/core/shared/utils/AllureUtils.ts`) is the fix: a small façade that wraps `allure-js-commons` so a spec writes one line instead of five, and gets a named, collapsible step with a real screenshot inside it regardless of outcome.

The screenshot displays the Allure Test body interface. On the left is a sidebar with navigation links: Overview, Categories, Suites (selected), Behaviors, Packages, Graphs, and Timeline. Below these are settings for Dark theme, Language (en), and a Collapse button. The main area is titled 'Allure - Test body' and shows a search bar with 'invalid credentials never reach'. Below the search bar is a table of Suites with columns for order, name, duration, status, and marks. The table lists several test suites, including 'chromium', 'firefox', 'Mobile Chrome', and 'Mobile Safari'. The 'chromium' suite is expanded, showing a test case '#3 invalid credentials never reach the authenticated area (authentication)' with a duration of 351ms. The right panel shows the details for this test case, including its severity (blocker), duration (351ms), parameters (Project: chromium), and execution steps. The test body is expanded, showing a timeline of steps: Before Hooks (8 sub-steps, 62ms), Navigate to login and attempt sign-in (7 sub-steps, 103ms), Screenshot (20ms), Verify access was denied (3 sub-steps, 24ms), Login attempt result (21.8 KiB, 149ms), and After Hooks (6 sub-steps, 149ms).

Severity, duration, and a full step timeline — generated by three `AllureUtils` calls in the spec.

The screenshot displays the Allure web interface. On the left is a sidebar with navigation links: Overview, Categories, Suites (selected), Behaviors, Packages, Graphs, and Timeline. Below these are settings for Dark theme, Language (en), and a Collapse button. The main area is titled 'Suites' and shows a search bar with 'invalid credentials never reach'. A table lists test suites with columns for order, name, duration, status, and marks. The table shows results for 'chromium', 'firefox', and 'Mobile Chrome' browsers, each with a 'Sample - Security Tests' sub-suite. The 'Sample - Security Tests' sub-suite for 'chromium' is expanded, showing a test case '#3 invalid credentials never reach the authenticated area (authentication)' with a status of 'Passed' and a duration of 351ms. The 'Test body' section is expanded, showing a 'Login attempt result' step with a screenshot of a 'Sign in' form. The form has fields for 'Username' (not a real user@example.com) and 'Password' (xxx), a 'Login' button, and an error message 'Invalid username or password'. The screenshot is inline with the test step, showing the exact state at the time it was taken.

The screenshot is inline, at the exact step it was taken — not bundled separately at the end.

The API

AllureUtils.setCategory(epic, feature, severity?)

Sets the labels a test is grouped and prioritized by in one call, instead of three separate `allure.epic/feature/severity` calls.

AllureUtils.step(name, fn)

Wraps a block of actions as one named, collapsible step — the building block for everything else here.

AllureUtils.steps(name, ...actions)

Same idea, for a flat list of actions run in sequence — skips writing `async () => { await a(); await b(); }` at the call site.

AllureUtils.attachScreenshot(page, name?)

Captures and attaches a PNG, regardless of pass or fail — this is what produced both screenshots above.

`AllureUtils.attachJson(name, data)`

Attaches any object as pretty-printed JSON — request payloads, computed values, anything worth inspecting later.

`AllureUtils.attachVideo(page, context, name?)`

Attaches the full recording for a test. Requires a context created with `recordVideo` set — see `e2e-tests.sample.spec.ts` for the complete pattern.

`AllureUtils.parameter(name, value)`

Shows a labelled value on the report page — durations, computed flags, anything worth surfacing without a full attachment.

Regenerate after every run. Raw results write automatically, but the static site does not rebuild itself — run `npm run allurereport` (or `allure:generate`) after the tests finish, every time, or you'll be looking at a stale report.

10

Visual regression

Pixel-level diffing, with automatic baseline bootstrap.

`src/core/tools/compare.js` diffs two PNGs with `pixelmatch` and fails past a configurable threshold. If no baseline exists yet, it creates one from the current screenshot and exits clean — the first run is never a false failure.

```
npm run compare -- baseline.png current.png diff.png --threshold=0.03
```

The same technique is also available as a typed, importable method —

`ScreenshotUtils.compare()` in `src/core/shared/utils/` — for specs that want the check without shelling out to a script.

Chatmodes & skills

Five personas you switch into, two behaviors Copilot applies on its own.

NAME	KIND	BEST FOR
 Healer	chatmode	Diagnose and fix a failing test
 Planner	chatmode	Draft a test plan before writing code
 Generator	chatmode	Write spec files and page objects
 API Testing	chatmode	Scaffold API and Pact contract tests
 Manual Testing	chatmode	Generate a manual QA checklist
 Code Review	skill • auto	Audit POM compliance, security, anti-patterns
 Test Debugging	skill • auto	Systematic failure triage

Activating a chatmode in VS Code

1

Open Copilot Chat

`Cmd/Ctrl+Alt+I`

2

Pick from the dropdown

Or skip it and prefix your message directly: `@healer Fix the failing test in tests/wesendcv.spec.ts`

3

Skills need nothing

Just ask a normal question — "review this spec for POM compliance" loads the Code Review skill automatically.

MCP server

Let an AI client run your tests as tool calls instead of shell commands.

`mcp/mcp-server.ts` exposes this framework over the Model Context Protocol (stdio transport). Any MCP-compatible client — Claude Desktop, Claude Code, VS Code — can launch it as a subprocess.

```
npm run mcp:server
```

TOOL	ARGUMENTS	DOES
<code>run_playwright_test</code>	<code>testFile</code> , <code>project?</code> , <code>headed?</code>	Runs one spec file
<code>run_all_tests</code>	—	Runs the full suite
<code>get_test_report</code>	—	Lists <code>playwright-report/</code> contents

CLIENT CONFIG

```
{
  "mcpServers": {
    "playwright-ai-agent": {
      "command": "npx",
      "args": ["tsx", "mcp/mcp-server.ts"],
      "cwd": "/absolute/path/to/repo"
    }
  }
}
```

Running in Docker

Same suite, containerized — for parity with CI.

```
docker compose up --build
```

Reports and results are bind-mounted back to the host, so `reports/` is populated exactly as a local run would leave it. Override the default command to run a specific suite:

```
docker compose run --rm playwright-tests npm run test:smoke
```

CI/CD

What happens on every push and PR.

`.github/workflows/ci.yml` runs on every push and pull request against `main` :

```
checkout → setup-node(20) → npm ci → lint → typecheck  
→ install:browsers → npm test → allure:generate → upload artifact
```

A manual `workflow_dispatch` trigger lets you pick the environment and browser for an on-demand run from the Actions tab.

Troubleshooting

The fast paths, in order of how often you'll need them.

A test failed

Check `reports/html` first — screenshot, video, and trace are already attached, plus the last 50 log lines.

Need to step through it

`npm run test:debug` for the Inspector, or `npm run test:ui` for the interactive runner.

Allure report looks stale

Regenerate — it never rebuilds itself. `npm run allurereport` .

Browser install issues

`npm run install:browsers` — safe to re-run any time.

Generated from the live repository — every command, file path, and screenshot on this page was run and captured directly against this codebase.